

semmcci: Monte Carlo Confidence Intervals

Ivan Jacob Agaloos Pesigan

Installation

You can install the CRAN release of `semmcci` with:

```
install.packages("semmcci")
```

You can install the development version of `semmcci` from [GitHub](#) with:

```
if (!require("pak")) install.packages("pak")
pak::pkg_install("jeksterslab/semmcci")
```

Documentation

See [GitHub Pages](#) for package documentation.

Description

In the Monte Carlo method, a sampling distribution of parameter estimates is generated from the multivariate normal distribution using the parameter estimates and the sampling variance-covariance matrix. Confidence intervals for defined parameters are generated by obtaining percentiles corresponding to $100(1 - \alpha)\%$ from the generated sampling distribution, where α is the significance level.

Monte Carlo confidence intervals for free and defined parameters in models fitted in the structural equation modeling package `lavaan` can be generated using the `semmcci` package. The package has two main functions, namely, `MC()` and `MCStd()`. The output of `lavaan` is passed as the first argument to the `MC()` function to generate Monte Carlo confidence intervals. Monte Carlo confidence intervals for the standardized estimates can also be generated by passing the output of the `MC()` function to the `MCStd()` function. A description of the package and code examples are presented in Pesigan and Cheung (2024).

Example

A common application of the Monte Carlo method is to generate confidence intervals for the indirect effect. In the simple mediation model, variable **X** has an effect on variable **Y**, through a mediating variable **M**. This mediating or indirect effect is a product of path coefficients from the fitted model.

```
library(semmcci)
library(lavaan)
```

Data

```
n <- 1000
X <- rnorm(n = n)
M <- 0.50 * X + rnorm(n = n)
Y <- 0.25 * X + 0.50 * M + rnorm(n = n)
data <- data.frame(X, M, Y)
```

Model Specification

The indirect effect is defined by the product of the slopes of paths **X** to **M** labeled as **a** and **M** to **Y** labeled as **b**. In this example, we are interested in the confidence intervals of `indirect` defined as

the product of **a** and **b** using the `:=` operator in the **lavaan** model syntax.

```
model <- "  
  Y ~ cp * X + b * M  
  M ~ a * X  
  indirect := a * b  
  direct := cp  
  total := cp + (a * b)  
"
```

Model Fitting

We can now fit the model using the `sem()` function from **lavaan**.

```
fit <- sem(data = data, model = model)
```

Monte Carlo Confidence Intervals

The `fit` **lavaan** object can then be passed to the `MC()` function to generate Monte Carlo confidence intervals.

```
MC(fit, R = 20000L, alpha = 0.05)  
  
#> Monte Carlo Confidence Intervals  
#>      est      se      R  2.5% 97.5%  
#> cp      0.2060 0.0346 20000 0.1389 0.2727  
#> b      0.5354 0.0309 20000 0.4738 0.5950  
#> a      0.4933 0.0316 20000 0.4311 0.5548  
#> Y~~Y    0.9343 0.0418 20000 0.8524 1.0161  
#> M~~M    0.9709 0.0436 20000 0.8846 1.0549
```

```
#> X~~X      0.9858 0.0000 20000 0.9858 0.9858
#> indirect 0.2641 0.0228 20000 0.2201 0.3093
#> direct    0.2060 0.0346 20000 0.1389 0.2727
#> total     0.4702 0.0352 20000 0.4008 0.5388
```

Standardized Monte Carlo Confidence Intervals

Standardized Monte Carlo Confidence intervals can be generated by passing the result of the `MC()` function to `MCStd()`.

Note: We recommend setting `fixed.x = FALSE` when generating standardized estimates and confidence intervals to model the variances and covariances of the predictors if they are assumed to be random.

```
fit <- sem(data = data, model = model, fixed.x = FALSE)
unstd <- MC(fit, R = 20000L, alpha = 0.05)
vcov(unstd)
```

#>	cp	b	a	Y~~Y	M~~M
#> cp	1.177315e-03	-4.700807e-04	2.202454e-06	6.083859e-06	2.230910e-05
#> b	-4.700807e-04	9.615005e-04	-6.549644e-06	-1.220359e-05	-1.518311e-05
#> a	2.202454e-06	-6.549644e-06	9.801450e-04	9.835835e-07	1.041514e-05
#> Y~~Y	6.083859e-06	-1.220359e-05	9.835835e-07	1.737269e-03	-5.061938e-07
#> M~~M	2.230910e-05	-1.518311e-05	1.041514e-05	-5.061938e-07	1.908664e-03
#> X~~X	7.346590e-06	-4.223766e-06	-7.779378e-07	3.258669e-06	-5.861026e-06
#> indirect	-2.308883e-04	4.708057e-04	5.218488e-04	-5.545961e-06	-1.783091e-06
#> direct	1.177315e-03	-4.700807e-04	2.202454e-06	6.083859e-06	2.230910e-05
#> total	9.464271e-04	7.250136e-07	5.240512e-04	5.378973e-07	2.052601e-05
#>	X~~X	indirect	direct	total	
#> cp	7.346590e-06	-2.308883e-04	1.177315e-03	9.464271e-04	

```
#> b      -4.223766e-06  4.708057e-04 -4.700807e-04  7.250136e-07
#> a      -7.779378e-07  5.218488e-04  2.202454e-06  5.240512e-04
#> Y~~Y    3.258669e-06 -5.545961e-06  6.083859e-06  5.378973e-07
#> M~~M   -5.861026e-06 -1.783091e-06  2.230910e-05  2.052601e-05
#> X~~X    1.950403e-03 -2.432393e-06  7.346590e-06  4.914196e-06
#> indirect -2.432393e-06  5.127753e-04 -2.308883e-04  2.818870e-04
#> direct   7.346590e-06 -2.308883e-04  1.177315e-03  9.464271e-04
#> total    4.914196e-06  2.818870e-04  9.464271e-04  1.228314e-03
```

MCStd(unstd)

```
#> Standardized Monte Carlo Confidence Intervals
#>      est      se      R  0.05%   0.5%   2.5%  97.5%  99.5% 99.95%
#> cp      0.1710 0.0282 20000 0.0782 0.0968 0.1149 0.2263 0.2431 0.2627
#> b      0.4926 0.0257 20000 0.4087 0.4251 0.4411 0.5416 0.5586 0.5752
#> a      0.4451 0.0253 20000 0.3600 0.3778 0.3943 0.4942 0.5092 0.5252
#> Y~~Y    0.6531 0.0243 20000 0.5675 0.5893 0.6036 0.6996 0.7146 0.7316
#> M~~M    0.8019 0.0225 20000 0.7242 0.7407 0.7558 0.8445 0.8573 0.8704
#> X~~X    1.0000 0.0000 20000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
#> indirect 0.2193 0.0173 20000 0.1660 0.1757 0.1860 0.2535 0.2658 0.2780
#> direct   0.1710 0.0282 20000 0.0782 0.0968 0.1149 0.2263 0.2431 0.2627
#> total    0.3903 0.0268 20000 0.3002 0.3199 0.3368 0.4419 0.4575 0.4747
```

References

Pesigan, I. J. A., & Cheung, S. F. (2024). Monte Carlo confidence intervals for the indirect effect with missing data. *Behavior Research Methods*, 56(3), 1678–1696. <https://doi.org/10.3758/s13428-023-02114-4>

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://www.R-project.org/>